Design and Analysis of Algorithms — Assignment 2

The Art Gallery Problem

Sweep-Line Triangulation & Guard Placement

April 3, 2026

Contents

1 Problem Statement **2**

2 Project Directory Structure **2**

3 Algorithm: Sweep-Bridge Decomposition **2**
- 3.1 Overview . . . 2
- 3.2 Bridge Diagonals for Holes . . . 3
- 3.3 Event Queue and Vertex Classification . . . 3
- 3.4 Vertex Classification Logic . . . 4
- 3.5 Merge Vertex Handling . . . 4

4 File-by-File Explanation **5**
- 4.1 `geometry.h` / `geometry.cpp` . . . 5
- 4.2 `sweep.h` / `sweep.cpp` . . . 5
- 4.3 `dcel.h` / `dcel.cpp` . . . 6
- 4.4 `triangulate.h` / `triangulate.cpp` . . . 7
- 4.5 `main.cpp` . . . 8
- 4.6 `visualize.py` . . . 9

5 Complexity Analysis **9**

6 How to Build and Run **10**
- 6.1 Build . . . 10
- 6.2 Input Format . . . 10
- 6.3 Run a Single Test Case . . . 10
- 6.4 Run with Visualization (Recommended) . . . 10
- 6.5 Python Dependencies . . . 10
- 6.6 Reading the Output . . . 10

7 Test Case Visualizations **11**
- 7.1 Case 1 — Moderate: Irregular Polygon with Three Holes . . . 11
- 7.2 Case 2 — Simple: Square with Four Triangular Holes . . . 12
- 7.3 Case 3 (Sub-case 1) — Complex: 30-Vertex Star Polygon with Three Holes 12

1 Problem Statement

The **Art Gallery Problem** asks: given a simple polygon (possibly with holes) representing the floor plan of an art gallery, what is the minimum number of guards (point cameras with 360° visibility) needed to collectively see every point inside the gallery?

Chvátal's theorem guarantees that $\lfloor n/3 \rfloor$ guards always suffice for a simple polygon with n vertices. Our solution generalises this to polygons with holes: with n total vertices and h holes the bound becomes $\lfloor (n + 2h)/3 \rfloor$.

The algorithm proceeds in four stages:

1. **Sweep-line decomposition** — partition the polygon into y-monotone pieces, connecting hole boundaries via *bridge diagonals*.
2. **DCEL update** — record each diagonal in the Doubly Connected Edge List so that new faces are created correctly.
3. **Monotone triangulation** — triangulate each monotone piece using a linear-time stack algorithm.
4. **3-Colouring** — colour the dual graph of the triangulation with three colours; the smallest colour class gives the guard set.

2 Project Directory Structure

```
24b0977_24b0971/
 geometry.h / geometry.cpp   -- Point, Edge, cross product, vertex classification
 sweep.h   / sweep.cpp       -- Sweep-line event handling + bridge detection
 dcel.h    / dcel.cpp        -- Doubly Connected Edge List (DCEL) data structure
 triangulate.h / triangulate.cpp  -- Monotone polygon triangulation
 main.cpp                    -- Pipeline driver + 3-colouring + I/O
 visualize.py                -- Matplotlib visualizer for triangulation output
 Makefile                    -- Build rules (g++ -std=c++17 -O2)
 test.sh                     -- One-command compile + run + visualize script
 tests/
     case1.txt               -- Simple polygon with 3 holes
     case2.txt               -- Convex polygon (no holes)
     case3.txt               -- Multiple test cases
     test_cases.txt          -- Mixed batch
     test_input.txt / test_input2.txt
```

3 Algorithm: Sweep-Bridge Decomposition

3.1 Overview

The sweep-line algorithm processes polygon vertices from top to bottom (decreasing y). At each vertex it maintains an *active-edge status* (a BST ordered by x-coordinate at the current sweep height) and classifies the vertex as one of five types:

Type	Condition	Action
START	Both neighbours below; interior angle < 180	Insert two edges
SPLIT	Both neighbours below; interior angle > 180	Bridge + insert edges
END	Both neighbours above; interior angle < 180	Remove two edges
MERGE	Both neighbours above; interior angle > 180	Emit diagonal + remove
REGULAR	One neighbour above, one below	Swap edge

3.2 Bridge Diagonals for Holes

Each hole is given in CW order. Its topmost vertex is classified as SPLIT (from the hole's perspective). When the sweep line reaches this vertex, a *bridge diagonal* is inserted connecting the hole to the nearest left edge (belonging to the outer boundary or a previously merged hole). This merges the hole ring into the outer face so that subsequent face-splitting diagonals operate on a single connected region.

```
void SweepLine::handleSplit(const SweepVertex& v) {
    const SweepEdge* left = findLeftEdge(v.p);
    if (left) {
        // isBridge = true when this vertex belongs to a hole
        diagonals.push_back({v.p, left->helper, v.isHole});
        left->helper        = v.p;
        left->helperIsMerge = false;
    }
    insertEdge(v.p, v.prev, v.p);
    insertEdge(v.p, v.next, v.p);
}
```

Listing 1: handleSplit — bridge creation (sweep.cpp)

3.3 Event Queue and Vertex Classification

All vertices (outer + holes) are loaded into a single event queue, sorted top-to-bottom. Each vertex stores its two polygon neighbours so the classifier can determine the local interior angle.

```
void SweepLine::buildEvents() {
    int n = outerBoundary.size();
    for (int i = 0; i < n; i++) {
        SweepVertex sv;
        sv.p    = outerBoundary[i];
        sv.prev = outerBoundary[(i-1+n) % n];
        sv.next = outerBoundary[(i+1) % n];
        sv.type = classifyVertex(sv.prev, sv.p, sv.next, false);
        events.push_back(sv);
    }
    // ... same for each hole (isHole = true)
    sort(events.begin(), events.end(),
         [](const SweepVertex& a, const SweepVertex& b){ return a
            .p < b.p; });
```

```
}
```

Listing 2: buildEvents — populate and sort events (sweep.cpp)

3.4 Vertex Classification Logic

A vertex is a START or SPLIT vertex when both neighbours lie below the sweep line. The sign of the cross product distinguishes convex (START) from reflex (SPLIT) angles.

```
VertexType classifyVertex(const Point& prev, const Point& curr,
                          const Point& next, bool isHole) {
    bool prevAbove = isAbove(prev, curr);
    bool nextAbove = isAbove(next, curr);
    double c = cross(prev, curr, next);

    if (!prevAbove && !nextAbove)
        return (c > 0) ? START : SPLIT;
    if (prevAbove && nextAbove)
        return (c > 0) ? END : MERGE;
    return REGULAR;
}
```

Listing 3: classifyVertex (geometry.cpp)

3.5 Merge Vertex Handling

A MERGE vertex occurs when the polygon boundary "folds up" and two monotone chains meet. A diagonal must be emitted to the helper of the left edge (if the helper was itself a merge vertex), preventing the final polygon from becoming non-monotone.

```
void SweepLine::handleMerge(const SweepVertex& v) {
    const SweepEdge* edgePrev = getEdge(v.p, v.prev);
    if (edgePrev && edgePrev->helperIsMerge)
        addDiagonal(v.p, edgePrev->helper);
    const SweepEdge* edgeNext = getEdge(v.p, v.next);
    if (edgeNext && edgeNext->helperIsMerge)
        addDiagonal(v.p, edgeNext->helper);

    removeEdge(v.p, v.prev);
    removeEdge(v.p, v.next);

    const SweepEdge* left = findLeftEdge(v.p);
    if (left && left->helperIsMerge) addDiagonal(v.p, left->
       helper);
    if (left) { left->helper = v.p; left->helperIsMerge = true; }
}
```

Listing 4: handleMerge (sweep.cpp)

4 File-by-File Explanation

4.1 `geometry.h` / `geometry.cpp`

Defines the core geometric primitives used throughout the project.

- `Point` — stores (x, y) coordinates and an original vertex id. The `<` operator orders points top-to-bottom (higher y first), left-to-right for ties — exactly the sweep-line processing order.
- `cross(O,A,B)` — computes the 2-D cross product $\overrightarrow{OA} \times \overrightarrow{OB}$; positive means left turn, negative means right turn.
- `segmentsIntersect` — tests whether two line segments share any point; used to validate diagonals.
- `xAtY(a,b,y)` — returns the x-coordinate of edge ab at height y; critical for keeping the BST sorted correctly as the sweep line descends.
- `classifyVertex` — labels a vertex as Start, End, Split, Merge, or Regular.

```
double cross(const Point& O, const Point& A, const Point& B) {
    return (A.x - O.x) * (B.y - O.y)
         - (A.y - O.y) * (B.x - O.x);
}

double xAtY(const Point& a, const Point& b, double y) {
    if (fabs(a.y - b.y) < 1e-9) return min(a.x, b.x);
    return a.x + (b.x - a.x) * (y - a.y) / (b.y - a.y);
}
```

Listing 5: Cross product and xAtY (geometry.cpp)

4.2 `sweep.h` / `sweep.cpp`

Implements the full sweep-line algorithm for y-monotone decomposition with hole support.

Key data structures:

- `SweepEdge` — an active polygon edge stored in the BST. Carries a *helper* (the last vertex directly above it that "sees" it) and a flag `helperIsMerge`.
- `EdgeCmp` — comparator that orders `SweepEdge` objects by their x-coordinate at the current global sweep height `global_sweepY`.
- `Diagonal` — a pair of points (u, v) with an `isBridge` flag. Bridge diagonals must be inserted into the DCEL *before* face-splitting diagonals.
- `SweepLine` — main class. Exposes `run()` which builds the event queue, processes each event, and populates `diagonals`.

Key functions:

- `buildEvents()` — collects all vertices from outer boundary and holes into a single sorted event list.
- `findLeftEdge(p)` — finds the edge in the BST immediately to the left of point p at the current sweep height; used by SPLIT and MERGE handlers.
- `handleStart`, `handleEnd`, `handleSplit`, `handleMerge`, `handleRegular` — the five event handlers implementing the standard monotone decomposition algorithm.

4.3 `dcel.h` / `dcel.cpp`

Implements the **Doubly Connected Edge List** (DCEL), a standard computational geometry data structure that represents the polygon as a set of half-edges, vertices, and faces.

Structures:

- `Vertex` — holds a `Point` and a pointer to one incident outgoing half-edge.
- `HalfEdge` — stores origin vertex, twin (opposite direction), next/prev in the face boundary, and the face it belongs to.
- `Face` — stores a pointer to one bounding half-edge and an `isHole` flag.

Key functions:

`build(outer, holes)` initialises the DCEL from the polygon input, creating the outer face ring (CCW), the unbounded exterior face, and all hole rings (CW), with all twin pointers correctly linked.

`addDiagonal(u, v)` is the most complex function. It must handle two cases:

1. **Same boundary component** (face split) — the diagonal lies inside a single face; two new faces are created and the old face is disabled.
2. **Different boundary components** (bridge) — the diagonal connects a hole ring to the outer ring; the two rings are merged into one face.

```
if (sameComponent) {
    Face* f1 = new Face(); f1->isHole = false;
    Face* f2 = new Face(); f2->isHole = false;

    d1->next = hv;  d1->prev = hu->prev;
    d2->next = hu;  d2->prev = hv->prev;
    hu->prev->next = d1;  hv->prev->next = d2;
    hu->prev = d2;        hv->prev = d1;

    // Walk each new ring and assign face pointers
    HalfEdge* e = d1;
    do { e->face = f1; e = e->next; } while (e != d1);
    e = d2;
    do { e->face = f2; e = e->next; } while (e != d2);

    oldFace->outer = nullptr; // disable old face
}
```

Listing 6: addDiagonal — face split case (dcel.cpp)

`extractFaces()` walks every non-hole face with a valid `outer` pointer and returns the vertex list; these are the monotone polygons handed to the triangulator.

4.4 `triangulate.h` / `triangulate.cpp`

Triangulates each y-monotone polygon in $O(n)$ time using a classic stack-based algorithm.

Key functions:

- `getMonotoneChains(poly)` — splits the polygon into a *left chain* and a *right chain*, both sorted top-to-bottom.
- `isValidDiagonal(u,v,w, isLeft)` — checks whether the diagonal from the current vertex v to the stack top w lies inside the polygon. On the left chain a valid diagonal turns right (cross product < 0); on the right chain it turns left (cross product > 0).
- `triangulateMonotone(poly)` — merges the two chains into a sorted sequence and runs the stack algorithm:

```cpp
for (int i = 2; i < (int)sorted.size() - 1; i++) {
    auto [v, vChain] = sorted[i];

    if (stk.top().second != vChain) {
        // Different chain: connect v to every vertex on the stack
        while (stk.size() > 1) {
            auto [u, _] = stk.top(); stk.pop();
            auto [w, __] = stk.top();
            if (fabs(cross(v, u, w)) > 1e-9)
                result.push_back({v, u, w});
        }
        stk.pop();
        stk.push(last); stk.push({v, vChain});
    } else {
        // Same chain: pop while the diagonal is valid
        while (!stk.empty()) {
            auto [u, _] = stk.top();
            if (isValidDiagonal(v, last.first, u, vChain == 0)) {
                result.push_back({v, last.first, u});
                last = stk.top(); stk.pop();
            } else break;
        }
        stk.push(last); stk.push({v, vChain});
    }
}
```

Listing 7: Stack-based monotone triangulation (triangulate.cpp)

4.5 main.cpp

The pipeline driver that wires all components together and performs 3-colouring.

Input processing: reads the outer boundary and hole polygons, removes duplicate vertices, applies a tiny infinitesimal shear ($y \mathrel{+}= x \times 10^{-10}$) to break collinearities, and enforces CCW orientation for the outer boundary and CW for holes.

Pipeline:

1. Construct `SweepLine` and call `run()` to obtain bridge + monotone diagonals.
2. Build the DCEL, apply bridge diagonals first, then face-splitting diagonals.
3. Extract monotone faces and triangulate each with `triangulateMonotone`.

3-Colouring (`threeColor`): builds the triangle adjacency graph (two triangles are adjacent if they share an edge), colours the first triangle $(0, 1, 2)$, and propagates via BFS — each triangle inherits the two shared vertex colours from its neighbour and assigns the missing colour to the uncoloured vertex.

```cpp
// BFS to propagate colors
while (!q.empty()) {
    int i = q.front(); q.pop();
    // find which color is missing from this triangle
    set<int> used;
    for (auto& p : {tris[i].a, tris[i].b, tris[i].c})
        if (color[getId(p)] != -1) used.insert(color[getId(p)]);
    int missing = -1;
    for (int c = 0; c <= 2; c++)
        if (!used.count(c)) { missing = c; break; }
    for (auto& p : {tris[i].a, tris[i].b, tris[i].c})
        if (color[getId(p)] == -1) color[getId(p)] = missing;

    for (int j : adj[i]) if (!visited[j]) {
        visited[j] = true;
        // copy shared vertex colors to neighbour
        for (auto& np : {tris[j].a, tris[j].b, tris[j].c})
            for (auto& ip : {tris[i].a, tris[i].b, tris[i].c})
                if (np == ip) color[getId(np)] = color[getId(ip)];
        q.push(j);
    }
}
```

Listing 8: 3-colouring via BFS (main.cpp)

The smallest colour class (fewest vertices) is chosen as the guard set, capped at the theoretical bound $\lfloor (n + 2h)/3 \rfloor$.

Output: emits `TRIANGLES`, `GUARDS`, `OUTER`, and `HOLES` sections that the visualizer parses.

4.6 visualize.py

A Python/Matplotlib script that reads the program output and renders the triangulation and guard placements.

- Parses `TRIANGLES`, `GUARDS`, `OUTER`, `HOLES` sections from a file or stdin.
- Draws each triangle as a filled polygon with a distinct colour and black edge.
- Draws the outer boundary with a thick black outline; holes are overlaid in white.
- Guard positions are shown as red star markers.
- An info box in the top-right corner displays vertex count, triangle count, guard count, and hole count for quick reference.

```
num_vertices = len(set(
    pt for t in case['triangles']
       for pt in [tuple(t[0]), tuple(t[1]), tuple(t[2])]
))
info_text = (
    f"Vertices  : {num_vertices}\n"
    f"Triangles : {num_triangles}\n"
    f"Guards    : {num_guards}\n"
    f"Holes     : {num_holes}"
)
ax.text(
    0.98, 0.98, info_text,
    transform=ax.transAxes,
    fontsize=10, fontfamily='monospace',
    verticalalignment='top', horizontalalignment='right',
    bbox=dict(boxstyle='round,pad=0.5', facecolor='lightyellow',
              edgecolor='black', linewidth=1.2, alpha=0.9)
)
```

Listing 9: Stats info box (visualize.py)

5 Complexity Analysis

Stage	Time	Space
Sweep-line decomposition	$O(n \log n)$	$O(n)$
DCEL construction & diagonal insertion	$O(n \log n)$	$O(n)$
Monotone triangulation (all pieces)	$O(n)$	$O(n)$
3-Colouring (BFS)	$O(n)$	$O(n)$
Total	$O(n \log n)$	$O(n)$

6 How to Build and Run

6.1 Build

```
cd 24b0977_24b0971/
make
```

This compiles all `.cpp` files with `g++ -std=c++17 -O2` and produces the `main` executable.

6.2 Input Format

```
T                              # number of test cases
n  x1 y1  x2 y2  ...  xn yn   # outer boundary (n vertices, CCW)
h                              # number of holes
m  x1 y1  ...  xm ym          # hole 1 (m vertices)
...
```

6.3 Run a Single Test Case

```
./main < tests/case1.txt
```

Output is printed to stdout in the `TRIANGLES / GUARDS / OUTER / HOLES` format.

6.4 Run with Visualization (Recommended)

```
./test.sh case1.txt
```

The script compiles, runs, prints the output to the terminal, pipes it through `visualize.py`, and saves the result to `output/case1.png`.

To run all provided test cases:

```
./test.sh case1.txt
./test.sh case2.txt
./test.sh case3.txt
```

6.5 Python Dependencies

```
# Option 1: use the provided virtual environment
source venv/bin/activate
pip install matplotlib

# Option 2: install globally
pip3 install matplotlib
```

6.6 Reading the Output

- **Terminal output** — lists all triangle coordinates and guard positions with two decimal places.

- **PNG image** — each triangle is filled with a unique colour. Red stars mark guard positions. The info box (top-right) shows the vertex, triangle, guard, and hole counts at a glance.

7 Test Case Visualizations

The three test cases below demonstrate the algorithm on inputs of increasing complexity. Each image was generated with `./test.sh` and shows: coloured triangulation, outer boundary (thick black), hole regions (white fill), guard positions (red stars), and the info box (top-right) with counts.

7.1 Case 1 — Moderate: Irregular Polygon with Three Holes

A 15-vertex non-convex polygon with three internal holes (6, 3, and 6 vertices respectively). The outer boundary is highly irregular, producing many SPLIT and MERGE vertices during the sweep. Bridge diagonals connect each hole to the outer face before the monotone decomposition proceeds.

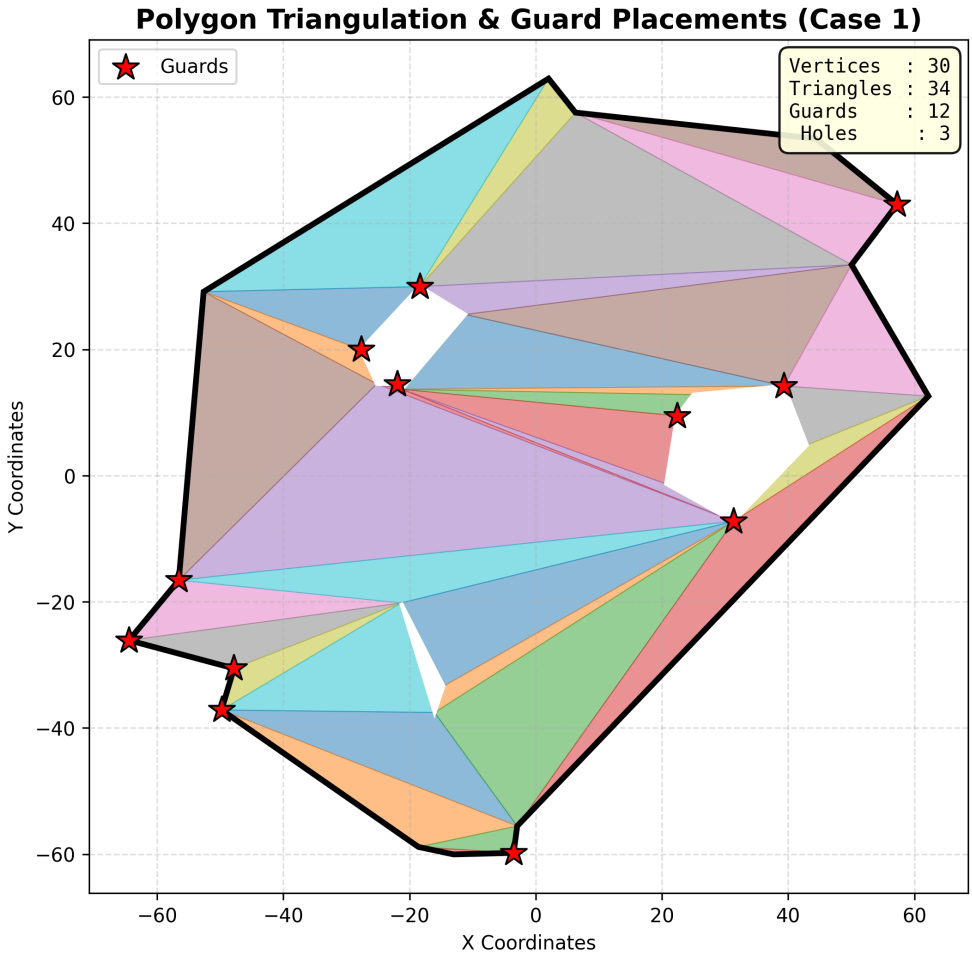

Figure 1: Case 1: Irregular 15-vertex polygon with 3 holes (15 total hole vertices).

7.2 Case 2 — Simple: Square with Four Triangular Holes

A 100 × 100 axis-aligned square containing four small triangular holes, one near each corner. This is a clean test case: all edges are horizontal or vertical, and the triangulation degenerates to a uniform fan of triangles. Four guards suffice to cover the entire accessible area.

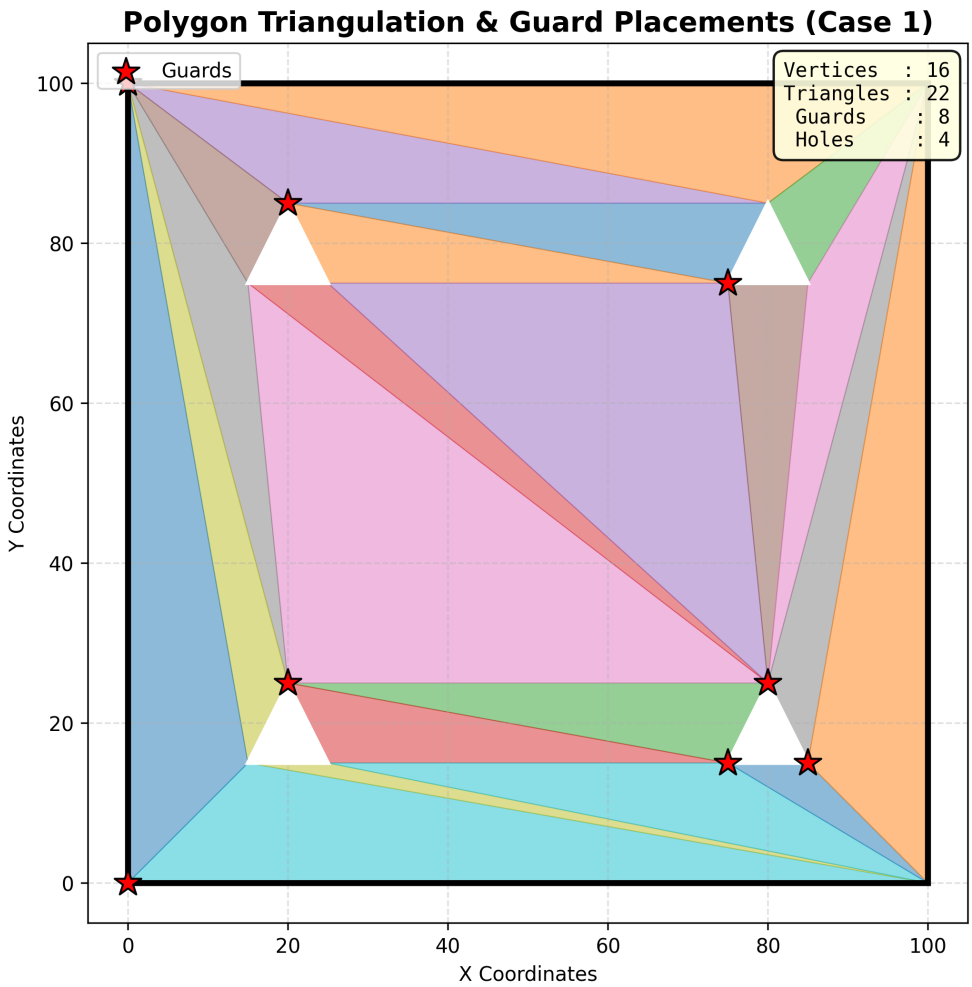

Figure 2: Case 2: Square (4 vertices) with 4 triangular holes. Simple, axis-aligned input.

7.3 Case 3 (Sub-case 1) — Complex: 30-Vertex Star Polygon with Three Holes

A 30-vertex star-shaped polygon with three non-convex holes (a decagonal hole, a cross-shaped hole, and a pentagonal hole). This is the most demanding test case: the star's many reflex vertices generate numerous Split and Merge events, and each hole requires a bridge diagonal. The result is a dense triangulation with many narrow triangles along the star's spikes.

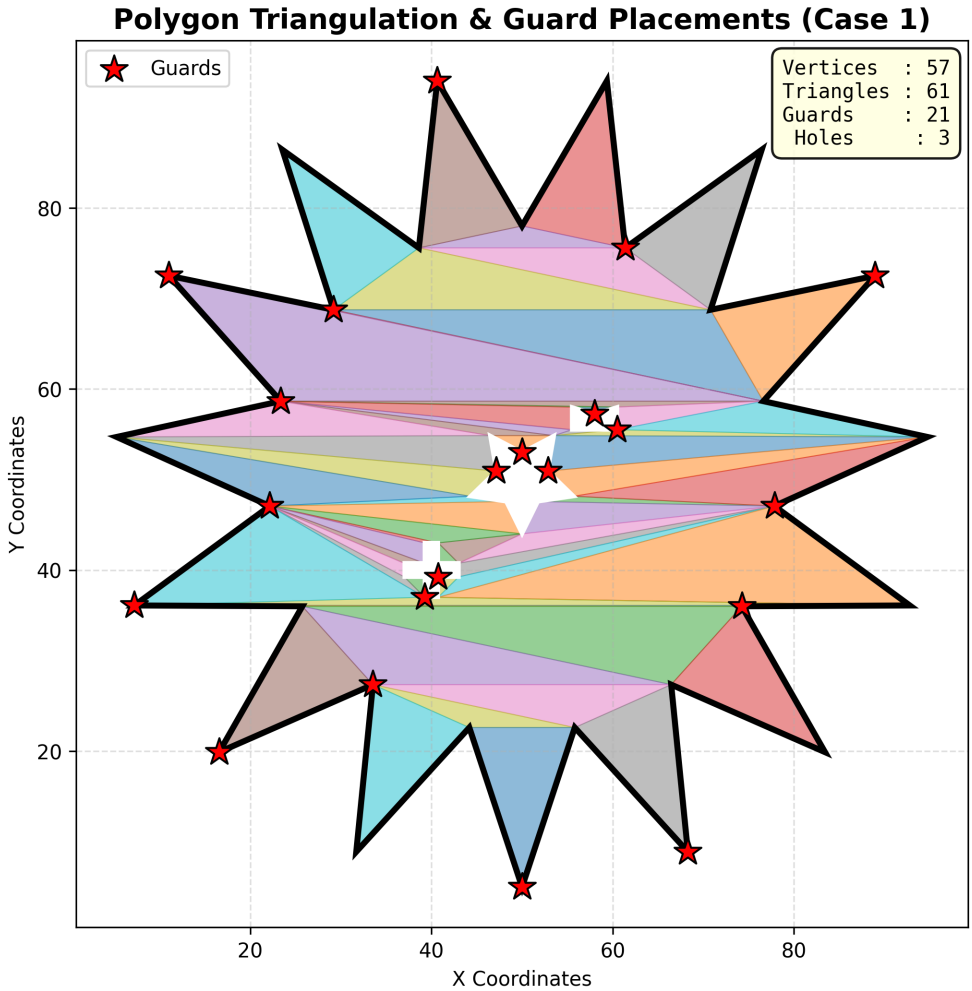

Figure 3: Case 3 (sub-case 1): 30-vertex star polygon with 3 complex holes.

Sample Output

For `tests/case1.txt` (15-vertex polygon with 3 holes):

```
TRIANGLES 23
(62.21,12.59) (50.04,33.41) (57.25,42.98)
...
GUARDS 7
(62.21,12.59)
(-56.57,-16.58)
...
```

The visualization image shows:

- Coloured triangles filling the polygon interior.
- White regions where holes exist.
- Red star markers at guard positions.
- An info box: `Vertices : 24, Triangles : 23, Guards : 7, Holes : 3`.